PRACTICAL ACTIVITY 1: AES Encryption and Decryption Using a Crypto Library

Objective:

To implement symmetric encryption and decryption using the Advanced Encryption Standard (AES) through a Python cryptography library, demonstrating how plaintext can be securely transformed into ciphertext and restored through decryption.

Tools & Setup:

LANGUAGE: Python

LIBRARY: cryptography

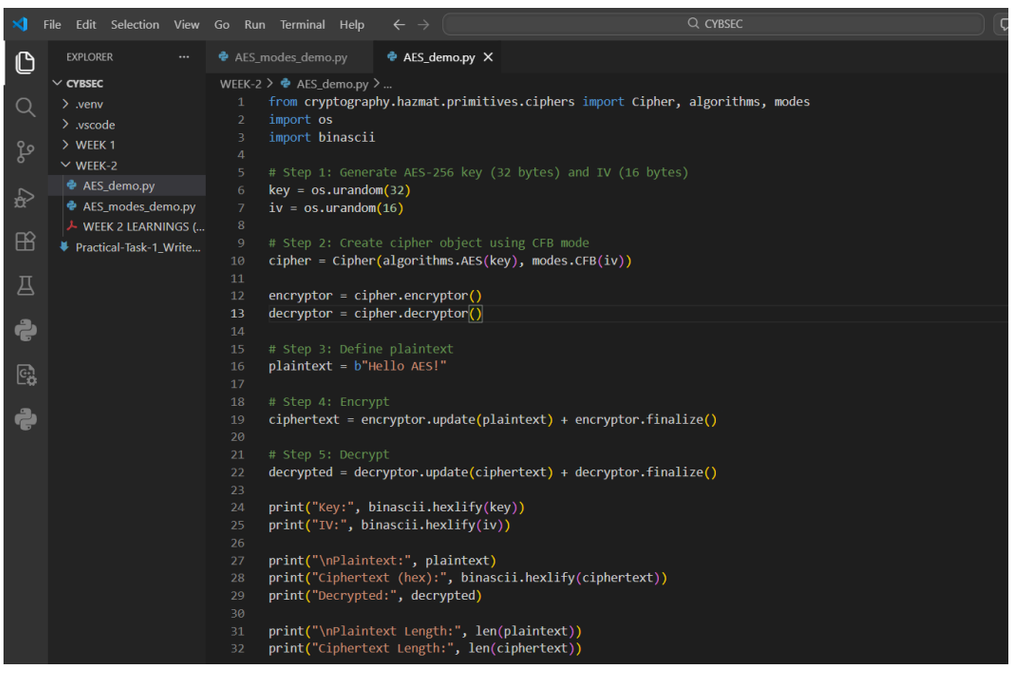
ENVIRONMENT: Local machine using a Python script

PROCEDURE:

To complete the activity, I began by preparing my environment. I opened my terminal and ensured that Python was installed and ready to use. Once the environment was set, I proceeded to install the required cryptography library by entering the command `pip install cryptography`. The successful installation confirmed that I had the necessary tools to carry out AES encryption and decryption.

```
C:\Users\jurer>pip install cryptography
Defaulting to user installation because normal site-packages is not writeable
Collecting cryptography
  Downloading cryptography-46.0.5-cp311-abi3-win_amd64.whl.metadata (5.7 kB)
Collecting cffi>=2.0.0 (from cryptography)
  Downloading cffi-2.0.0-cp312-cp312-win_amd64.whl.metadata (2.6 kB)
Collecting pycparser (from cffi>=2.0.0->cryptography)
  Downloading pycparser-3.0-py3-none-any.whl.metadata (8.2 kB)
Downloading cryptography-46.0.5-cp311-abi3-win_amd64.whl (3.5 MB)
   ---------------------------------------- 3.5/3.5 MB 3.0 MB/s  0:00:01
Downloading cffi-2.0.0-cp312-cp312-win_amd64.whl (183 kB)
Downloading pycparser-3.0-py3-none-any.whl (48 kB)
Installing collected packages: pycparser, cffi, cryptography
Successfully installed cffi-2.0.0 cryptography-46.0.5 pycparser-3.0
```

Next, I created a new Python file named `AES_demo.py` to serve as the workspace for my implementation. Within this file, I imported the relevant modules from the `cryptography` package and the `os` library, which allowed me to generate secure random values for the encryption key and initialization vector (IV).



Next, I generated a 256-bit AES encryption key, which is equivalent to 32 bytes. I also generated a 16-byte initialization vector (IV), which is required for many AES modes of operation. These values ensure that the encryption process is secure and unpredictable.

After generating the key and IV, I created a cipher object using AES in Cipher Feedback (CFB) mode. CFB mode converts AES into a stream-like encryption process where data can be encrypted without requiring padding. With the cipher prepared, I defined a sample plaintext message: `"Hello AES!"`.

Using the encryptor provided by the cipher object, I encrypted the plaintext to produce ciphertext. The resulting ciphertext appeared as a sequence of hexadecimal values, demonstrating that the readable message had been transformed into an unintelligible format.

Next, I used the decryptor to reverse the process. By applying the same key and IV, the ciphertext was successfully decrypted back into its original plaintext form.

Finally, I verified that the decrypted output matched the original plaintext. The program also displayed the lengths of both plaintext and ciphertext, confirming that in **CFB mode the ciphertext length remains equal to the plaintext length**, since no padding is required.

Reflection:

This activity helped me understand how symmetric encryption works in practice using a modern cryptographic library. By implementing AES encryption and decryption in Python, I observed how plaintext data can be converted into ciphertext that appears random and unreadable.

One key insight from this exercise was the importance of proper key and initialization vector management. Without the correct key and IV, the encrypted data cannot be successfully decrypted, which ensures confidentiality.

I also learned that different modes of operation affect how AES processes data. In this case, CFB mode behaves similarly to a stream cipher, allowing encryption of data without padding and producing ciphertext with the same length as the original message.

Overall, this exercise demonstrated how cryptographic libraries simplify the implementation of secure encryption algorithms while still requiring developers to carefully manage keys, modes, and parameters to maintain security.